DETERMINISTIC RULE ENGINES, FINANCIAL CALCULATIONS, AND EXPLAINABLE BUSINESS LOGIC

Engineering Case Study for Affordable Housing and Regulated Enterprise Systems

Author	Venkata Lakshmi Parimala Pasupuleti
Engineering focus	Affordable Housing, data-intensive backend systems, production ownership, AI-assisted operations
Business areas	Eligibility, income limits, rent, HAP, tax credits, certification, documents
Document type	Technical engineering case study
Implementation boundary	Production experience and clearly labeled modernization guidance
Core principle	Probabilistic AI may assist; deterministic logic remains authoritative
Status	Portfolio documentation based on real system situations

This document describes system-level engineering patterns grounded in the Affordable Housing modules, data workflows, migrations, and production investigations I owned or supported. It does not claim legal interpretation authority or ownership of every enterprise calculation engine.

Document Map

1. Executive Summary
2. Scope and Domain Context
3. Why Deterministic Rules Matter
4. Decision Stack and Separation of Responsibilities
5. Authoritative Inputs and Effective Context
6. Rule Representation and Versioning
7. Eligibility Decision Workflow
8. Income-Limit Resolution
9. Rent, Gross-Rent, and HAP Calculations
10. Tax-Credit and Set-Aside Logic
11. Configuration Precedence and Overrides
12. Calculation Precision, Rounding, and Boundaries
13. Explainability and Decision Records
14. Validation and Testing Strategy
15. Database and Stored-Procedure Design
16. Release, Migration, and Rollback
17. Production Observability and Incident Investigation
18. AI Assistance and Safety Boundary
19. Performance and Scalability
20. Security and Auditability
21. Failure Modes and Risk Register
22. Architecture Trade-offs
23. Modernization Roadmap
24. Reusable Engineering Checklists
25. Lessons Learned

1. Executive Summary

Affordable Housing software turns federal and program rules, annual reference data, household facts, property configuration, and time-sensitive certification context into eligibility, rent, HAP, tax-credit, and document outcomes. Because these outcomes are regulated and financially significant, the core decision path must be deterministic, versioned, reproducible, and explainable.

My production experience reinforced that correctness depends on more than a formula. The system must select the correct jurisdiction, household size, AMI percentage, effective year, program, property setting, and rule version before calculation begins. A technically correct formula applied to the wrong context is still a wrong result.

This case study describes how to structure deterministic business logic, separate authoritative calculation from AI assistance, preserve historical correctness, validate changes, and operate the system safely in production. It draws on annual income-limit processing, eligibility dependencies, RabbitMQ-based gross-rent work, document generation, database migration, and the LeaseI support-investigation architecture.

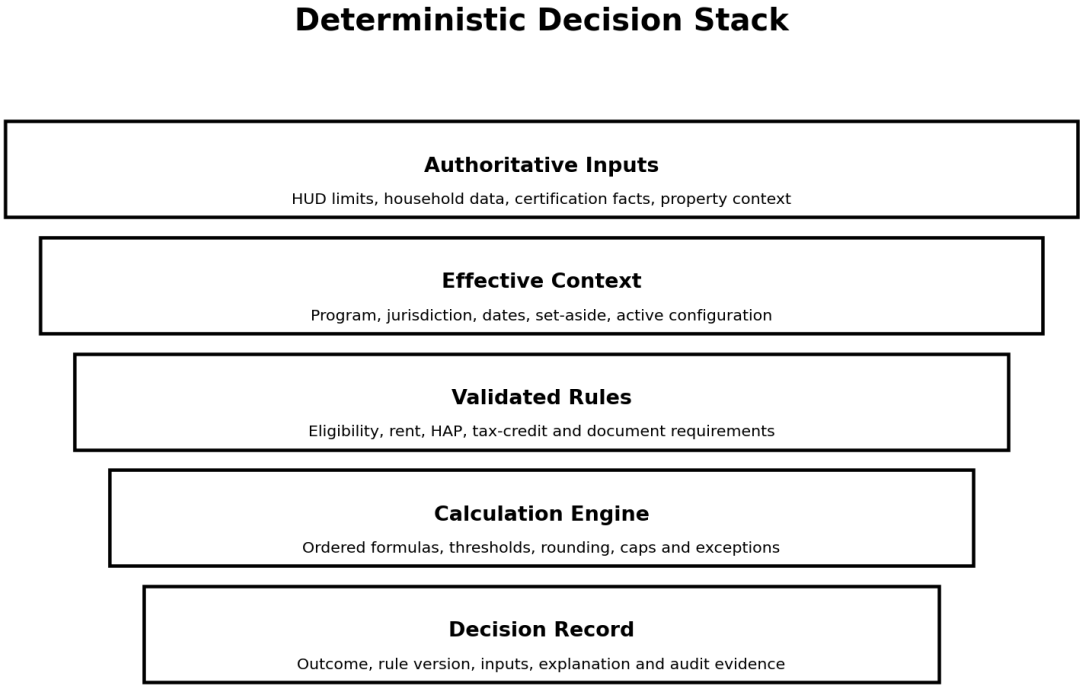


Figure 1. Deterministic decisions are built from authoritative inputs, effective context, versioned rules, calculation, and retained evidence.

2. Scope and Domain Context

The document focuses on the engineering design of business rules and calculations. It does not replace regulatory or legal interpretation. Product and compliance specialists define or approve the business meaning; engineering translates the approved behavior into versioned, testable, operable software.

Domain area	Representative inputs	Representative outputs
Income limits	Jurisdiction, household size,	Published threshold records and

Domain area	Representative inputs	Representative outputs
	year, AMI percentage, program	effective values
Eligibility	Household income, household size, certification date, program, property settings	Eligible, ineligible, conditional, or review-required state
Rent and gross rent	Program rules, unit context, limits, utility allowances, effective date	Maximum rent and related financial values
HAP	Tenant contribution, contract rent, subsidy context, program rules	Housing-assistance payment calculation
Tax credits and set-asides	Property election, unit designation, AMI restrictions, household result	Compliance classification and permitted outcome
Documents	Decision outputs, household/property data, template version, merge fields	Customer-facing forms, notices, and evidence

3. Why Deterministic Rules Matter

A deterministic rule engine produces the same outcome when given the same versioned inputs, configuration, and rule set. That property is essential for regulated workflows because decisions must be reproducible during audits, customer support, data corrections, and production incidents.

- Regulated decisions require consistency across users and time.
- Financial calculations need controlled precision and rounding.
- Historical certifications must remain reproducible after annual updates.
- Support teams need to explain which rule, setting, and value produced the outcome.
- Testing requires known expected outputs and stable boundary behavior.
- AI-generated interpretations cannot substitute for authoritative rule execution.

Deterministic does not mean simple. A rule engine may include complex branching, precedence, exceptions, effective dates, and dependencies. The key is that those choices are explicit and version-controlled rather than inferred unpredictably at runtime.

4. Decision Stack and Separation of Responsibilities

A maintainable decision architecture separates data acquisition, context resolution, rule selection, calculation, and explanation. Combining all concerns into one stored procedure or service may work initially, but it becomes harder to test, reason about, and evolve.

Layer	Responsibility	Typical failure if mixed
Input acquisition	Load household, property, program, reference, and configuration data	Hidden dependency or stale value

Layer	Responsibility	Typical failure if mixed
Context resolution	Choose jurisdiction, effective date, program, set-aside, and active overrides	Correct formula applied to wrong context
Rule selection	Resolve the applicable version and ordered rule set	Inconsistent historical behavior
Calculation	Apply thresholds, formulas, caps, rounding, and exceptions	Untraceable financial discrepancy
Decision persistence	Record result, rule version, inputs, and evidence	Cannot reproduce or explain outcome
Presentation	Translate decision into UI, API, document, or support explanation	Display logic changes business meaning

5. Authoritative Inputs and Effective Context

The rule engine should consume normalized, validated inputs rather than raw documents or untrusted free text. Inputs may come from annual HUD data, customer records, certification workflows, property configuration, or approved program reference tables.

Context dimension	Engineering requirement
Tenant and property	Scope every lookup and setting to the authorized customer context
Program	Select the correct HUD, LIHTC, or product program behavior
Jurisdiction	Resolve the correct geographic area before limit lookup
Household size	Use validated household composition at the certification point
Effective date	Select the rules, limits, and settings active for the determination date
AMI or set-aside	Apply the correct percentage and property election
Configuration	Resolve global, module, program, and property overrides deterministically

Effective context should be materialized into a decision input object or equivalent internal structure. That makes downstream rule execution independent from repeated ad hoc lookups and creates a stable audit snapshot.

6. Rule Representation and Versioning

Rules can be implemented in application code, stored procedures, decision tables, or configuration. The correct representation depends on complexity, performance, ownership, and change frequency. Regardless of representation, every consequential rule needs a stable identity and version.

Rule metadata	Purpose
rule_id	Stable business identity independent of implementation location
version	Distinguish behavior changes over time
effective_start / end	Select historically correct behavior
source_reference	Link to approved regulation, policy, or product requirement
owner	Identify the responsible domain and engineering owner
priority / order	Make execution sequence explicit
test_set	Link representative and boundary scenarios
status	Draft, approved, active, superseded, or retired

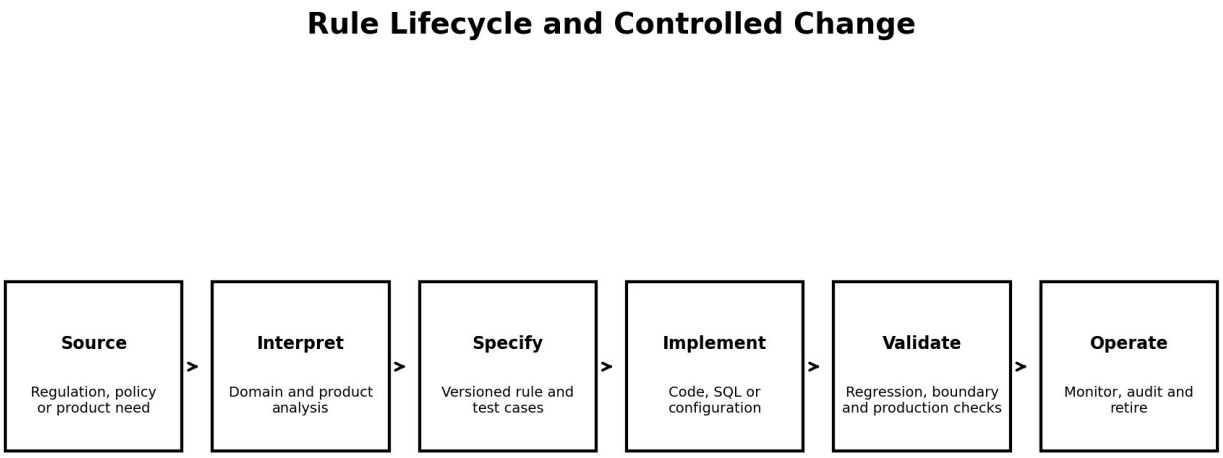


Figure 2. Rule changes move from authoritative source through interpretation, specification, implementation, validation, and operation.

A rule change should not silently overwrite history. New behavior should be introduced as a new version with an effective date. Existing historical decisions continue to reference the previous version.

7. Eligibility Decision Workflow

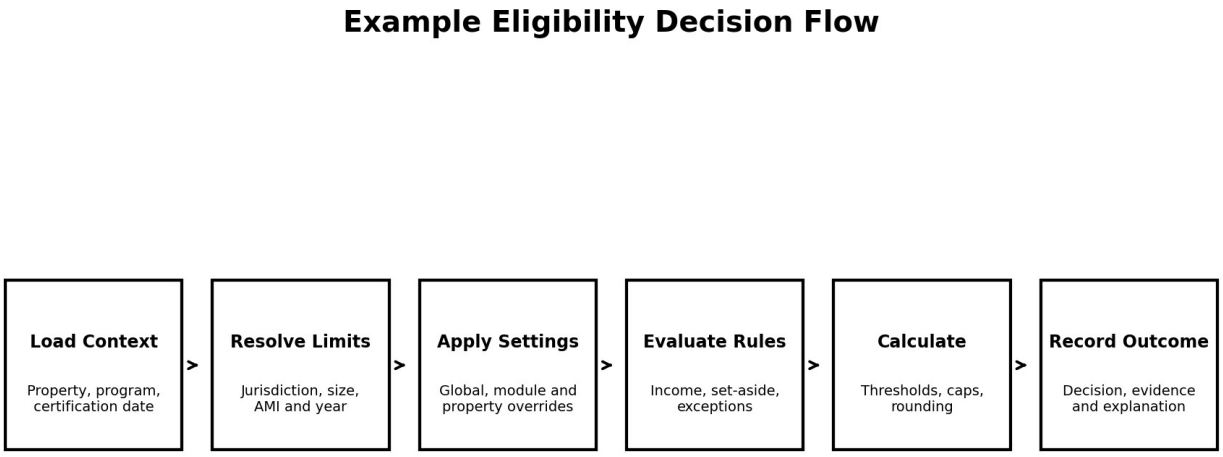


Figure 3. Eligibility is a context-resolution and rule-execution workflow, not one isolated comparison.

The workflow begins by resolving the property, program, jurisdiction, certification date, household, and applicable settings. It then retrieves the correct annual limits and evaluates ordered eligibility rules. The output should identify the final state and the evidence that produced it.

Decision state	Meaning
Eligible	All required deterministic checks passed
Ineligible	A defined rule failed with a recorded reason
Conditional	Additional product or program action is required
Review required	The system lacks authoritative input or encounters a controlled exception

Review-required is an important safe state. When required context is missing or contradictory, the engine should not guess. It should stop, preserve evidence, and route the case to an authorized user.

8. Income-Limit Resolution

Income-limit resolution is a foundational dependency. The engine must select the correct record by jurisdiction, household size, program, AMI percentage, effective year, and date. The annual pipeline transformed a few thousand source rows into approximately 180,000 domain-specific records, which made validation and effective-date selection critical.

- Validate source provenance and publication year.
- Map geographic identifiers consistently.
- Expand household-size and percentage variants deterministically.
- Prevent duplicate business keys and overlapping active ranges.

- Publish only after reconciliation and representative validation.
- Preserve prior years for historical certifications.

A live HUD API can be used as an authoritative lookup tool in a support or AI workflow, but the product calculation path still needs controlled availability, version selection, caching policy, and reconciliation with the application's published data.

9. Rent, Gross-Rent, and HAP Calculations

Rent-related calculations combine external limits, property and unit context, utility or allowance inputs, tenant contribution, subsidy rules, caps, and effective dates. These workflows are financially sensitive and often require batch processing across many tenant records.

Calculation concern	Engineering control
Input completeness	Reject or review when required financial or unit context is missing
Order of operations	Define the exact sequence for limits, allowances, contributions, caps, and subsidy
Precision	Use fixed decimal types rather than binary floating point
Rounding	Apply business-approved rounding at the defined stage
Batch processing	Use transaction boundaries and idempotent worker behavior
Recalculation	Record why and under which version a value was recalculated
Downstream impact	Identify documents, ledgers, notices, and reports affected by a change

RabbitMQ was used for large annual gross-rent workloads because the work could be processed asynchronously rather than blocking an interactive user request. The deterministic calculation remained inside the worker; the queue changed execution and scaling, not the business meaning.

10. Tax-Credit and Set-Aside Logic

Tax-credit and set-aside behavior introduces property-level elections and unit or household restrictions that may interact with general income and rent rules. The system needs explicit precedence and effective dating so a property configuration cannot silently change historical meaning.

- Represent property elections and set-aside percentages as governed configuration.
- Validate that the rule applies to the correct property, program, unit, and date.
- Record the selected restriction in the decision context.
- Separate general regulatory thresholds from property-specific elections.
- Expose conflicts or incomplete configuration as review-required conditions.

11. Configuration Precedence and Overrides

Configuration is runtime behavior. The nearly 24-hour eligibility investigation demonstrated that correct Affordable Housing rules can still produce an unexpected outcome when a system-level setting from another team overrides module behavior.

Scope	Example role	Precedence requirement
Global	Enterprise default or platform feature behavior	Lowest or explicitly defined base
Module	Affordable Housing-specific behavior	Overrides global where approved
Program	Program-specific requirements	Overrides module for the applicable program
Property	Customer/property election or exception	Overrides broader defaults within allowed boundaries
Operational	Temporary controlled mitigation	Highest but time-bounded and audited

Precedence must be deterministic and explainable. The decision record should show the candidate values, selected value, source scope, effective date, and reason. Hidden or undocumented precedence is a major source of support complexity.

12. Calculation Precision, Rounding, and Boundaries

Financial calculations require explicit numeric behavior. The implementation should define data type, scale, intermediate precision, rounding mode, and where rounding occurs. Different rounding stages can produce different final values even when the same formula is used.

Boundary category	Representative tests
Threshold equality	Exactly equal to an income or rent limit
One-unit boundary	One cent or one dollar below and above the threshold
Household-size edge	Minimum, maximum, and unusual household sizes
Effective-date edge	Day before, day of, and day after a new rule becomes active
Missing or zero values	Required missing input, permitted zero, or invalid negative value
Rounding edge	Values exactly at half increments and cumulative intermediate rounding

Rounding should be a named business rule, not an incidental language default. The rule version and tests should preserve the approved behavior across application and database implementations.

13. Explainability and Decision Records

Explainability for deterministic systems means exposing the exact inputs, rule version, context, calculation steps, and failed or passed conditions. This is different from asking an AI model to create a plausible explanation after the fact.

Decision-record field	Purpose
decision_id	Stable reference for support, documents, and audit
subject context	Tenant, property, household, unit, and certification identifiers
effective context	Program, jurisdiction, dates, AMI/set-aside, active configuration
input snapshot	Authoritative values used by the engine
rule versions	Exact rules and calculation version executed
step results	Passed, failed, skipped, and exception states
final outcome	Decision or financial values
explanation codes	Stable machine-readable reasons for UI and support
correlation and release	Link to workflow, deployment, and operational evidence

Documents and support tools should consume explanation codes and structured evidence rather than reimplementing decision logic. This prevents the explanation layer from drifting away from the actual engine.

14. Validation and Testing Strategy

Testing should combine unit-level rule verification, data-level reconciliation, workflow integration, migration validation, and production checks.

Test layer	Focus
Rule unit tests	Each condition, exception, threshold, and ordering rule
Decision-table tests	Combinations of program, context, settings, and boundaries
Calculation tests	Precision, rounding, caps, and known examples
Database tests	Stored procedures, constraints, effective dates, and transaction behavior
Integration tests	Reference lookup, configuration, eligibility,

Test layer	Focus
	documents, and messaging
Regression tests	Known production incidents and historical expected outcomes
Migration tests	Old/new result comparison and historical reproducibility
Production validation	Representative authorized cases and reconciliation after release

The 24-hour setting incident should become a permanent regression scenario: all Affordable Housing inputs pass, but a specific system setting changes the result. That test protects against repeating the same cross-team dependency failure.

15. Database and Stored-Procedure Design

Stored procedures can be appropriate for large set-based transformations and data-intensive calculations because they execute close to the data and support strong transaction boundaries. They also introduce maintainability and deployment trade-offs.

Benefit	Trade-off and control
Set-based performance	Complex SQL requires execution-plan analysis and representative tests
Transactional consistency	Long transactions can increase locks; use bounded work and checkpoints
Centralized data logic	Ownership can blur; document boundaries and version procedures
Reusable calculation path	Application and database releases must remain compatible
Auditable affected rows	Scripts still need review, expected scope, and rollback

Business logic should live in the database when set-based execution, integrity, or transaction locality provides real value. It should remain in application code when domain composition, testability, external tools, or independent scaling are more important. The decision should be explicit rather than ideological.

16. Release, Migration, and Rollback

Rule and calculation changes are high-risk because they can alter large numbers of customer outcomes without causing obvious technical errors. Release planning should include result comparison, effective-date behavior, data migration, document impact, queued work, and rollback state.

- Version rule behavior before deployment.
- Compare old and new outcomes across representative and boundary datasets.
- Deploy backward-compatible schema and contracts before cleanup.

- Capture pre-release counts, active versions, and database evidence.
- Validate both technical health and business outcomes after deployment.
- Define rollback for code, rule versions, data publication, queue messages, and generated documents.
- Preserve the release decision and validation results for later support investigations.

17. Production Observability and Incident Investigation

Observability should reveal which rule context and values produced an outcome. Generic application logs are insufficient for complex regulated decisions.

Signal	Operational value
Decision count by outcome	Detect unexpected shifts after a release or annual update
Rule-version distribution	Confirm the intended version is active
Missing-context rate	Identify data and integration problems
Configuration override rate	Reveal unexpected property or global behavior
Calculation duration	Detect performance regressions
Recalculation and regeneration count	Measure downstream recovery work
Support reason codes	Connect customer issues to deterministic failure categories

Incident investigation should start by reconstructing the decision context. In the setting incident, the rules and limits were correct; the missing evidence was the active configuration. A complete decision record would have surfaced that dependency much earlier.

18. AI Assistance and Safety Boundary

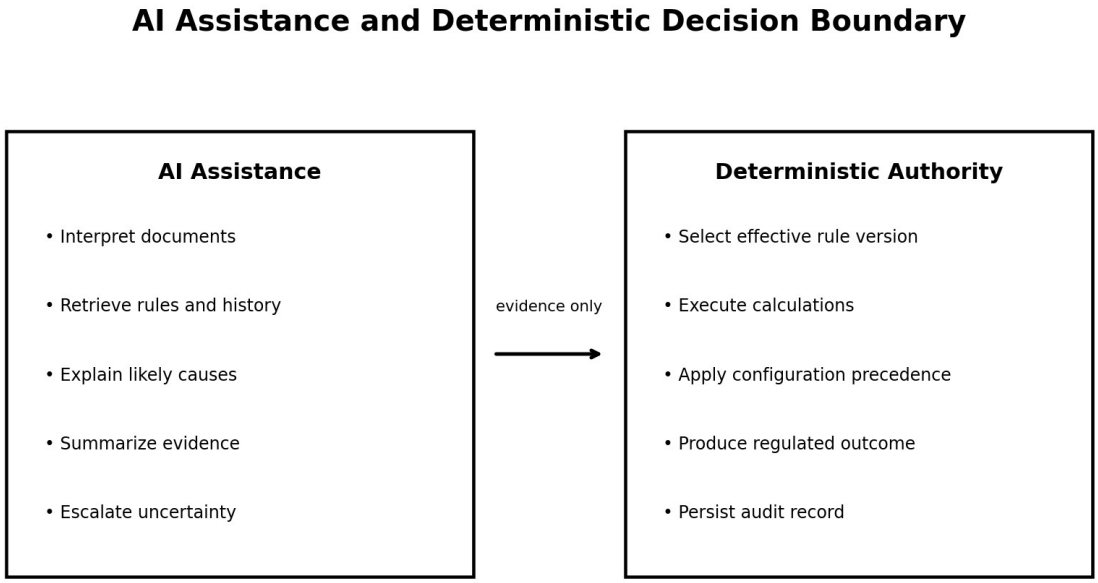


Figure 4. AI can interpret and connect evidence, while deterministic systems remain responsible for regulated outcomes.

LeaseI is designed to reduce investigation time by connecting product knowledge, regulatory text, historical incidents, and live structured data. It should explain why the deterministic system behaved as it did, not independently determine eligibility or financial values.

- AI may classify a support issue and identify the affected module.
- AI may retrieve regulatory text, product documentation, release history, and similar incidents.
- Approved read-only tools may fetch current HUD limits and property settings.
- A validation layer should require evidence for consequential claims.
- Low-confidence or conflicting cases should produce a human escalation packet.
- The deterministic engine and authorized staff remain the source of truth.

This boundary is enforced in the current LeaseI tenant self-service flow, which does not auto-resolve eligibility, configuration, or calculation questions regardless of confidence.

19. Performance and Scalability

Rule engines scale through efficient context retrieval, controlled database access, set-based operations, caching of safely versioned reference data, and asynchronous processing for high-volume recalculation.

Scaling concern	Design response
Repeated reference lookup	Cache by immutable version and effective context, with explicit invalidation
Large annual transformations	Stage, batch, index, reconcile, and publish atomically

Scaling concern	Design response
High-volume recalculation	Use RabbitMQ workers with idempotency and bounded concurrency
Database contention	Short transactions, appropriate indexes, plan monitoring, and partitioned work
Explanation overhead	Persist compact structured reason codes and generate presentation on demand
Historical growth	Retain versioned records with indexing and archival strategy

Performance optimization must preserve semantic correctness. A faster lookup that ignores effective date or property scope is not an optimization; it is a defect.

20. Security and Auditability

- Restrict decision inputs and records by tenant, property, and authorized role.
- Protect household, income, financial, and document data from unnecessary logging.
- Use distinct application, worker, migration, and support identities.
- Record configuration and rule changes with old value, new value, actor, reason, and effective date.
- Preserve immutable or tamper-evident evidence for high-impact changes.
- Keep support and AI diagnostic tools read-only by default.
- Audit access to decision records, generated documents, and privileged exports.

Auditability should be designed into the decision path. Reconstructing a result from incomplete logs after an incident is slower and less reliable than persisting the versioned context at decision time.

21. Failure Modes and Risk Register

Risk	Impact	Primary control
Wrong effective limit or rule	Incorrect eligibility or financial result	Versioned effective dating and reconciliation
Hidden configuration override	Unexpected cross-module behavior	Deterministic precedence and explainable configuration resolution
Rounding inconsistency	Financial discrepancy across systems	Named rounding rule and cross-layer tests
Duplicate asynchronous calculation	Repeated financial or document effect	Idempotency and transactionally recorded workflow state
Rule update without history	Cannot reproduce past decisions	Immutable rule versions and decision references
Incomplete input treated as valid	False decision instead of safe exception	Review-required state and mandatory context validation
Explanation drift	Support message differs from	Structured explanation codes

Risk	Impact	Primary control
	actual engine	derived from execution
AI overreach	Unsupported regulated decision	Read-only tools, evidence validation, human escalation

22. Architecture Trade-offs

Stored procedures versus application code

Stored procedures provide strong set-based performance and locality, while application code offers clearer domain composition and independent deployment. Use the representation that best matches the workload and ownership model.

Flexible configuration versus predictability

Configuration accelerates customer-specific behavior but can hide dependencies. Limit what may be configured, define precedence, version changes, and monitor override use.

Precomputed results versus real-time calculation

Precomputation improves response time but creates freshness and invalidation responsibilities. Real-time calculation improves immediacy but can increase latency and database load.

Detailed decision records versus storage and privacy

Rich evidence improves support and auditability but increases sensitive-data retention. Store the minimum structured context required to reproduce and explain the outcome.

Rule-engine generalization versus domain clarity

A highly generic engine can reduce duplication but may obscure domain-specific behavior. Generalize stable execution mechanics, not every business concept.

23. Modernization Roadmap

Phase	Focus	Outcome
1. Standardize context	Typed decision input, stable reason codes, rule and configuration versions	Decisions become reproducible and explainable
2. Strengthen tests	Decision tables, boundary cases, incident regressions, cross-layer parity	Lower release and calculation risk
3. Govern configuration	Precedence service, effective dates, ownership, change audit	Hidden dependencies become visible
4. Improve observability	Decision metrics, rule-version views, calculation traces	Faster detection and investigation
5. Modularize rule execution	Separate context resolution, rules, calculation, and	Safer independent evolution

Phase	Focus	Outcome
	presentation	
6. Add AI assistance	Evidence retrieval, explanation, and escalation around the deterministic engine	Reduced support effort without delegating authority
7. Automate controlled change	Versioned rule publication, migration gates, rollback validation	More reliable annual and policy updates

24. Reusable Engineering Checklists

24.1 New or changed rule

- Identify authoritative source, owner, effective date, and affected programs.
- Define stable rule ID, version, order, inputs, outputs, and reason codes.
- Create representative, boundary, missing-input, and historical tests.
- Assess downstream calculations, documents, reports, and support behavior.
- Define migration, rollout, monitoring, and rollback.

24.2 Financial calculation

- Use approved decimal precision and rounding mode.
- Define order of operations and intermediate rounding points.
- Validate caps, floors, negative values, missing values, and thresholds.
- Persist calculation version and explanation details.
- Test cross-language or database/application parity where both exist.

24.3 Production investigation

- Reconstruct tenant, property, program, date, household, and active settings.
- Retrieve the exact rule, limit, and calculation version.
- Compare expected and actual step results.
- Search recent configuration and release changes.
- Preserve evidence before correction.
- Convert the root cause into a regression test and durable knowledge entry.

24.4 AI-assisted explanation

- Retrieve only authorized evidence.
- Use read-only validated tools for current values.
- Reference the deterministic decision record.
- Label hypotheses and contradictions clearly.
- Escalate when evidence is incomplete or the decision is high impact.

25. Lessons Learned

- Correct formulas are not enough; correct context selection is equally important.
- Effective dating and versioning are core requirements for regulated decisions.

- Configuration must be governed and explained like code.
- Financial precision and rounding should be explicit business rules.
- Every decision should be reproducible from retained inputs, rules, and settings.
- Asynchronous execution changes scale and failure handling, not business meaning.
- Production incidents reveal missing dependencies that should become tests and knowledge.
- AI is most valuable when it retrieves and explains evidence around deterministic systems.
- Senior ownership includes design, validation, release, support, and long-term maintainability.

Closing Perspective

My experience in Affordable Housing taught me to treat business rules and financial calculations as long-lived production systems rather than isolated conditional statements. The strongest implementation is one that is correct, versioned, testable, observable, explainable, and safe to change. AI can make that system easier to understand and support, but the regulated decision remains grounded in deterministic logic and authoritative data.